# MICRO PROJECT:

# REPORT

**Domain -** OTT Services

**Assigned Topic -** Programme Catalogue

**One-Line Topic Brief -** Maintains content metadata, genres, episodes, and availability.

Jyotiraditya Parida
BA2025017

## 1. Implementation of the project-

The project was implemented as a Python app that connects to the MySQL server using the mysql.connector module. After connecting, it checks whether the 'myottdb' database exists. If not, it initializes the database by executing SQL scripts for schema, constraints, and data.

The app is driven by a menu which allows the user to interact with the database using simple inputs. The system constructs SQL queries dynamically for CRUD operations on any table.

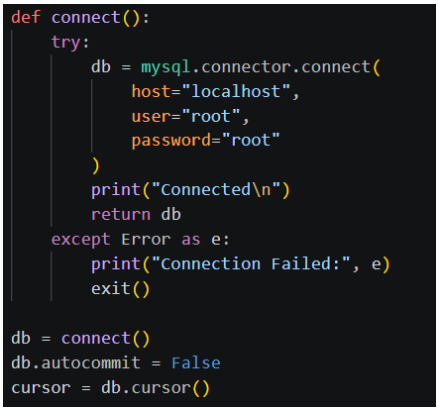
The Read, Update and Delete operations support selecting all columns or specific columns and allow the use of a WHERE clause with conditions such as =, <, <=, >, >=, LIKE.

The system supports transaction processing using commit and rollback, allowing the user to confirm or cancel changes. Error handling is implemented to manage connection failures, invalid inputs and SQL errors, ensuring the program does not crash and provides appropriate error messages to the user.

## 2. Features implemented in the project-

i) Connector function-

Connects Python to MySQL and handles connection errors.



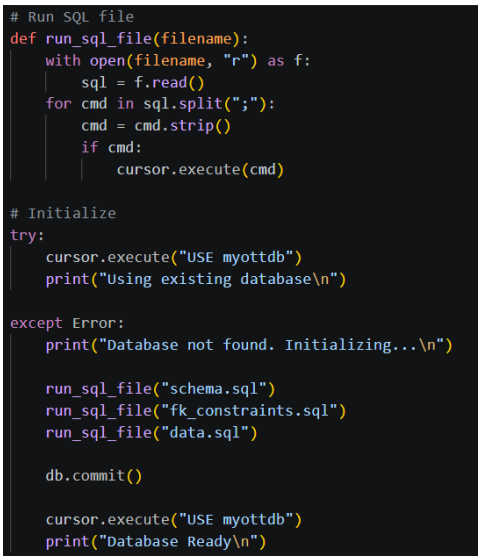
```
def connect():
    try:
        db = mysql.connector.connect(
            host="localhost",
            user="root",
            password="root"
        )
        print("Connected\n")
        return db
    except Error as e:
        print("Connection Failed:", e)
        exit()

db = connect()
db.autocommit = False
cursor = db.cursor()
```

ii) Database Initialization-

Checks if ‘myottdb’ database exists. If not, creates the database, tables, constraints, and inserts data using SQL files-

- schema.sql
- data.sql
- fk_constraints.sql



```
# Run SQL file
def run_sql_file(filename):
    with open(filename, "r") as f:
        sql = f.read()
    for cmd in sql.split(";"):
        cmd = cmd.strip()
        if cmd:
            cursor.execute(cmd)

# Initialize
try:
    cursor.execute("USE myottdb")
    print("Using existing database\n")

except Error:
    print("Database not found. Initializing...\n")

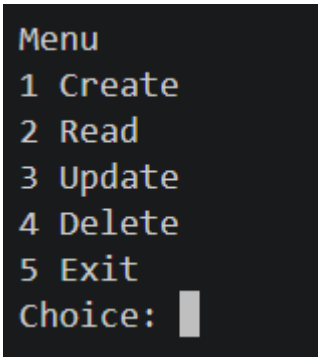
    run_sql_file("schema.sql")
    run_sql_file("fk_constraints.sql")
    run_sql_file("data.sql")

    db.commit()

    cursor.execute("USE myottdb")
    print("Database Ready\n")
```

iii) Menu Interface-

Provides options for Create, Read, Update and Delete.



```
Menu
1 Create
2 Read
3 Update
4 Delete
5 Exit
Choice:
```

iv) Create Operation-

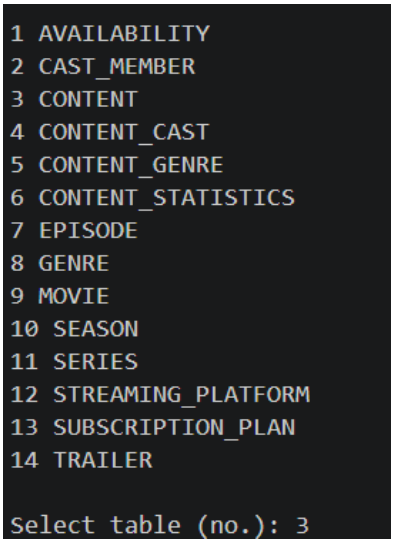
Allows user to add records to any table.

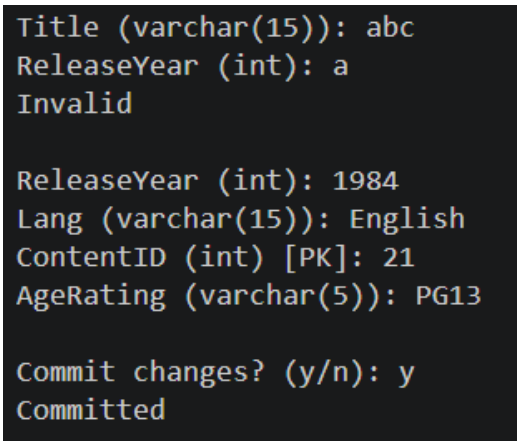
First, it shows all table names and asks user to select one. If an invalid table number is entered, an error message is shown.

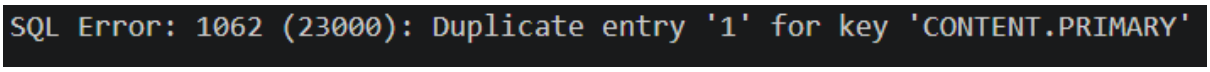
Then, it prompts the user to enter the attribute values according to the data type. If the user enters an attribute with the wrong data

type, for example, char instead of int, then the program gives an error message and prompts the user again. It also gives an error message if any constraint, for example, primary key constraint, is violated.

It also asks the user if they want to commit the changes or rollback.




v) Read Operation-

Allows user to retrieve data from any table.

First, it asks the user to choose a table. It gives an error message if the table doesn't exist.

Then, it asks which columns are to be displayed (* for all).

After that, it asks whether the WHERE clause is to be used to put conditions on the data. If not, it displays all rows.

If the WHERE clause is added, the program asks for the column name, the operator (=, <, <=, >, >=, LIKE) and the value.

```
Select table (no.): 3

Columns:
- Title
- ReleaseYear
- Lang
- ContentID
- AgeRating

SELECT columns (* for all OR c1, c2, ...)
Columns: *

Add WHERE clause? (y/n)
n

1 ('Inception', 2010, 'English', 1, 'PG13')
2 ('Dark', 2017, 'German', 2, '18+')
3 ('MoneyHeist', 2017, 'Spanish', 3, '18+')
4 ('Interstellar', 2014, 'English', 4, 'PG13')
5 ('StrangerThings', 2016, 'English', 5, '16+')
6 ('BreakingBad', 2008, 'English', 6, '18+')
7 ('Loki', 2021, 'English', 7, 'PG13')
8 ('TheOffice', 2005, 'English', 8, '13+')
9 ('Narcos', 2015, 'Spanish', 9, '18+')
10 ('Witcher', 2019, 'English', 10, '18+')
11 ('Friends', 1994, 'English', 11, '13+')
12 ('Avatar', 2009, 'English', 12, 'PG13')
13 ('Joker', 2019, 'English', 13, '18+')
14 ('Tenet', 2020, 'English', 14, 'PG13')
15 ('PeakyBlinders', 2013, 'English', 15, '18+')
16 ('Sherlock', 2010, 'English', 16, '16+')
17 ('Lucifer', 2016, 'English', 17, '18+')
18 ('Matrix', 1999, 'English', 18, '18+')
19 ('Titanic', 1997, 'English', 19, 'PG13')
20 ('Deadpool', 2016, 'English', 20, '18+')
```

```
Select table (no.): 3

Columns:
- Title
- ReleaseYear
- Lang
- ContentID
- AgeRating

SELECT columns (* for all OR c1, c2, ...)
Columns: Title, ReleaseYear

Add WHERE clause? (y/n)
y
WHERE (column) (condition =, <, <=, >, >=, LIKE) (value)
Column: Title
Operator: like
Value: S%

1 ('StrangerThings', 2016)
2 ('Sherlock', 2010)
```

vi) Update Operation-

Allows the user to update rows in any table.

First, it asks the user to choose a table. It gives an error message if the table doesn't exist.

Then, it asks which attribute/column is to be updated in the rows and asks for the new value.

After that, it asks whether the WHERE clause is to be used to put conditions on the rows. If not, it updates all the rows.

If the WHERE clause is added, the program asks for the column name, the operator (=, <, <=, >, >=, LIKE) and the value.

Finally, it asks the user if they want to commit the changes or rollback.

```
Select table (no.): 3

Columns:
- Title
- ReleaseYear
- Lang
- ContentID
- AgeRating

UPDATE (column) SET (value)
Column: ContentID
Value: 21

Add WHERE clause? (y/n)
y
WHERE (column) (condition =, <, <=, >, >=, LIKE) (value)
Column: Title
Operator: =
Value: Deadpool

Commit changes? (y/n): y
Committed
```

vii) Delete Operation-

Allows the user to delete rows in any table.

First, it asks the user to choose a table. It gives an error message if the table doesn't exist.

Then, it asks whether the WHERE clause is to be used to put conditions on the rows. If not, it asks if the user wants to delete all the rows. If yes, it deletes all rows.

If the WHERE clause is added, the program asks for the column name, the operator (=, <, <=, >, >=, LIKE) and the value.

Finally, it asks the user if they want to commit the changes or rollback.

```
Select table (no.): 3

Columns:
- Title
- ReleaseYear
- Lang
- ContentID
- AgeRating

Add WHERE clause? (y/n)
y
WHERE (column) (condition =, <, <=, >, >=, LIKE) (value)
Column: Title
Operator: =
Value: Deadpool

Commit changes? (y/n): y
Committed
```

```
Select table (no.): 3

Columns:
- Title
- ReleaseYear
- Lang
- ContentID
- AgeRating

Add WHERE clause? (y/n)
n
Delete ALL rows? (y/n): n
Cancelled
```